



Earnings Calendar — Technical Documentation

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Earnings Calendar — Technical Documentation	1
1. Purpose and scope	3
2. Architecture overview	3
3. URL, route, and deep links	4
4. Dependencies	5
5. User Interface (UI) components	6
6. Session state	8
7. Data layer	9
8. Page load and data fetch	11
9. Filter pipeline	11
10. Event click and detail flow	12
11. Email alert preferences	13
12. FullCalendar event shape	13
13. Error handling	14
14. Authentication and access control	15
15. Performance notes	15
16. Related files	15
17. Key code references	16

Application: Market Data Portal (MDP)

Page module: `app/pages/earnings_calendar.py` (~2,127 lines)

URL route: `/earnings_calendar`

Registered in: `app/main.py` as `( "pages/earnings_calendar.py", "Earnings Calendar", "earnings_calendar", False)`

1. Purpose and scope

The Earnings Calendar page is a **FullCalendar**-powered view of upcoming and historical earnings announcements across the Market Data Portal (MDP) universe. It also overlays **completed mergers and acquisitions (M&A)** closing events from key developments data.

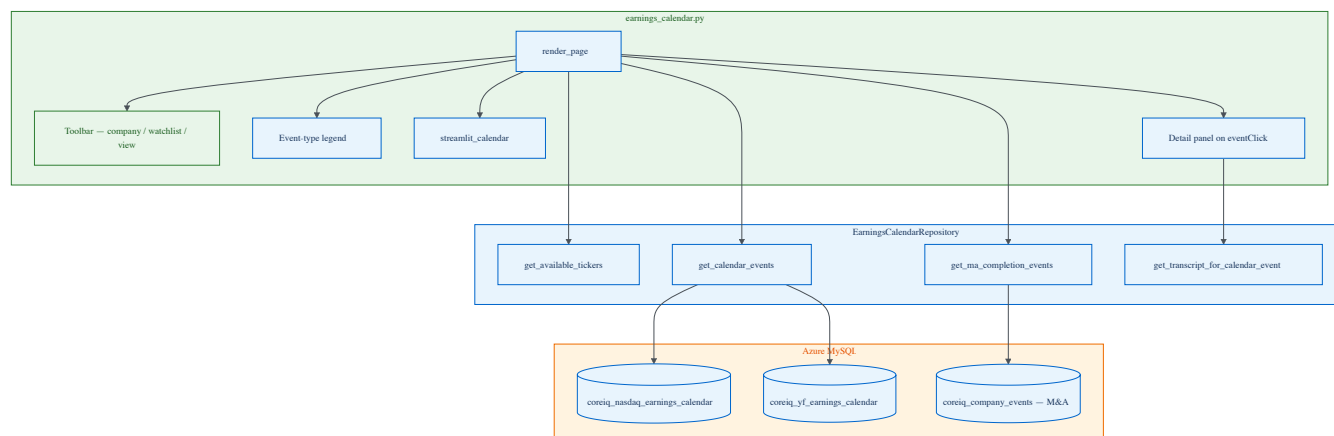
Core capabilities:

- **Month view** (`dayGridMonth`) and **Year view** (`multiMonthYear`) with list-month toggle
- **Company filter** (All Companies or one ticker) with cross-page ticker synchronization via `? ticker=`
- **Watchlist filter** (from Screening watchlists) with ticker + company-name disambiguation
- **Event-type legend** — clickable chips for Fiscal Quarter 1–4 (Q1–Q4) and M&A Completion; combines with company/watchlist filters
- **Event detail panel** — Earnings per share (EPS) actual/forecast, surprise %, market cap, investor relations (IR) webcast link, deep link to Earnings Calls transcript
- **Email alert preferences** — modal dialog to schedule reminders (companies, sectors, or watchlist scope)
- **Parallel data prefetch** on page load (tickers, date range, all events, M&A events)

2. Architecture overview

Top-down: the page orchestrates toolbar filters, prefetches repository data in parallel, converts rows to FullCalendar JSON, and renders `streamlit_calendar` with click-to-detail.

Top-down: the page orchestrates toolbar filters, prefetches repository data in parallel, converts rows to FullCalendar JSON, and renders streamlitcalendar with click-to-detail.



3. URL, route, and deep links

Entry	Behavior
<code>/earnings_calendar</code>	Default — All Companies, month view
<code>/earnings_calendar?ticker=AAPL</code>	Validates ticker via <code>validate_and_get_ticker</code> ; dropdown still defaults to All Companies but URL ticker is preserved on company change
Navigation bar	<code>navigation.py</code> links with <code>?ticker={active_ticker}</code>
Detail panel → transcript	<code>/earnings_calls?ticker={T}&year={Y}&quarter=Q{Q}&from=calendar</code>
Earnings Calls → calendar	Reverse deep link documented in earnings-calls.md

4. Dependencies

4.1 Python modules

Module	Role
<code>components/styles.py</code>	<code>render_styles()</code> , <code>hide_sidebar()</code>
<code>components/navigation.py</code>	<code>render_header()</code> , <code>render_coresight_footer()</code>
<code>core/auth_manager.py</code>	<code>require_auth()</code> optional; cookie hydrate via <code>main.py</code>
<code>data/repository.py</code>	<code>CompanyRepository</code> , <code>EarningsCalendarRepository</code>
<code>data/watchlist_service.py</code>	<code>get_user_watchlists</code> , <code>get_watchlist_companies</code>
<code>utils/ticker_utils.py</code>	<code>validate_and_get_ticker</code> , <code>DEFAULT_FALLBACK_TICKER</code>
<code>utils/earnings_alert_email.py</code>	<code>send_earnings_alert_preferences_confirmation</code>
<code>utils/server_logger.py</code>	Structured logging, <code>log_timing</code> , <code>new_rerun_id</code>
<code>streamlit_calendar</code>	FullCalendar widget (<code>st_calendar</code>)

4.2 External libraries

Library	Use
FullCalendar (via <code>streamlit-calendar</code>)	Month grid, multi-month year, list view
ThreadPoolExecutor	Parallel prefetch of 4 independent DB calls

5. User Interface (UI) components

5.1 Toolbar

Control	Session key	Notes
Email Alerts button	<code>ec_open_email_alerts</code>	Opens <code>@st.dialog</code> modal
Alert status pill	—	HTML badge: Alerts on / Alerts off
Company selectbox	<code>ec_company_filter</code>	"All Companies" or "Name (TICK)" labels
Watchlist selectbox	<code>ec_watchlist_filter</code>	Disabled when user has no watchlists
Month Wise / Year Wise	<code>ec_view</code>	Toggles <code>calendar</code> vs <code>year</code> FullCalendar mode

5.2 Event-type legend

Five clickable chips (Q1, Q2, Q3, Q4, M&A Completion) using Coresight brand colors from `_KIND_COLORS`. Toggling a chip updates `ec_active_kinds`, clears `ec_selected_event`, and increments `ec_cal_version` to remount FullCalendar.

5.3 FullCalendar views

Mode	<code>initialView</code>	Key options
Month	<code>dayGridMonth</code>	<code>height: 680</code> , <code>dayMaxEvents: 4</code> , list-month toggle
Year	<code>multiMonthYear</code>	<code>multiMonthMaxColumns: 3</code> , <code>dayMaxEvents: 3</code>

Callbacks restricted to `["eventClick"]` only — avoids `eventsSet` rerun storms.

5.4 Detail panel

Shown in right column when `ec_selected_event` is set:

- **Earnings events** — EPS actual/forecast, surprise %, fiscal quarter end, market cap, report time, transcript link, IR webcast link
- **M&A events** — acquirer, target, deal type, transaction value, announce/close dates, source link

Close button clears selection and bumps `ec_cal_version`.

5.5 Email alerts dialog

`@st.dialog("Email alerts for upcoming earnings calls", width="large")` — coverage modes:

1. **Select Companies** — all calendar companies or specific multiselect
2. **Select Sectors** — all sectors or specific multiselect (matched in-memory against ticker metadata)
3. **Select Watchlist** — one watchlist; encoded in database (DB) as sentinel `watchlist_id:<id>`

Reminders sent at **08:00 UTC**; `days_before` 0–60.

6. Session state

Key	Type	Purpose
<code>ec_view</code>	<code>str</code>	"calendar" or "year"
<code>ec_selected_event</code>	<code>dict</code> \ <code>None</code>	FullCalendar event object from click
<code>ec_cal_version</code>	<code>int</code>	Bumped to remount calendar widget (new <code>key</code>)
<code>_ec_current_date</code>	<code>str</code> (ISO)	<code>initialDate</code> for FullCalendar — updated on event click
<code>ec_active_watchlist_id</code>	<code>int</code> \ <code>None</code>	Active watchlist filter
<code>ec_active_watchlist_name</code>	<code>str</code>	Display name for caption
<code>ec_active_kinds</code>	<code>list[str]</code>	Active legend kinds: Q1–Q4, <code>ma</code>
<code>ec_company_filter</code>	<code>str</code>	Selectbox label value
<code>ec_watchlist_filter</code>	<code>str</code>	Watchlist selectbox label
<code>ec_alert_send_email</code>	<code>bool</code>	Checkbox in dialog
<code>ec_alert_days_before</code>	<code>int</code>	Number input
<code>ec_alert_company_scope</code>	<code>str</code>	Radio: Companies / Sectors / Watchlist
<code>ec_alert_save_in_progress</code>	<code>bool</code>	Disables Save during persist
<code>ec_alert_save_success</code>	<code>bool</code>	Flash success on next dialog open
<code>ec_alert_save_mailed</code>	<code>bool</code>	Whether confirmation email sent
<code>_active_page</code>	<code>str</code>	Used to reset filters on fresh navigation
<code>active_ticker</code>	<code>str</code>	Cross-page ticker sync

Reset on fresh page entry (`_active_page != "earnings_calendar"`): clears `ec_company_filter` , `ec_watchlist_filter` , watchlist ids, resets `ec_active_kinds` to all.

7. Data layer

7.1 Primary tables

Table	Source	Content
<code>coreiq_nasdaq_earnings_calendar</code>	Nasdaq ingest	SEC-lineage earnings dates, EPS, fiscal quarter ending
<code>coreiq_yf_earnings_calendar</code>	Yahoo Finance (YF) ingest	Non-Nasdaq / supplemental earnings
<code>coreiq_company_events</code>	Key developments	M&A Closing rows (<code>event_subtype = 'M&A Closing'</code> , <code>ma_is_closed = 1</code>)
<code>coreiq_companies</code>	Company master	Names, sectors for filters
<code>coreiq_ir_websites</code>	IR URLs	Webcast links joined by ticker + source (SEC vs YFinance)
<code>coreiq_earnings_alert_preferences</code>	Alert prefs	Per-user email reminder configuration
<code>coreiq_earnings_alert_dedupe</code>	Dispatcher	Prevents duplicate sends
<code>coreiq_earnings_alert_delivery_log</code>	Dispatcher	Delivery audit trail
<code>coreiq_av_earnings_call_transcripts</code>	Transcripts	Mapped via <code>get_transcript_for_calendar_event</code>

7.2 EarningsCalendarRepository key methods

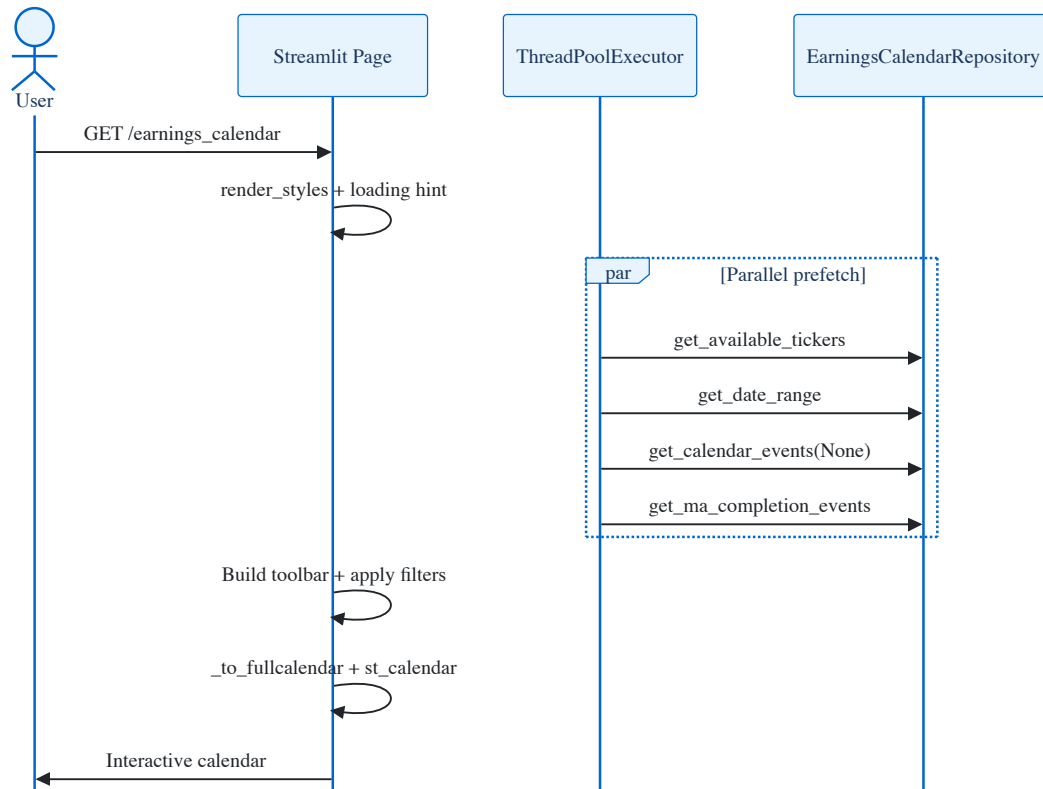
Method	Cache TTL	Description
<code>get_available_tickers()</code>	300s	Dropdown metadata: name, ticker, sector
<code>get_date_range()</code>	300s	Min/max earnings dates across both feeds
<code>get_calendar_events(tickers)</code>	300s	Union Nasdaq + YF, deduped per <code>(ticker, fiscal_quarter_ending)</code> — prefers latest <code>fetch_at_utc</code> , then Nasdaq over YF
<code>get_ma_completion_events()</code>	300s	M&A completion overlay from <code>coreiq_company_events</code>
<code>get_transcript_for_calendar_event(ticker, date)</code>	600s	Maps calendar date → transcript year/quarter
<code>get_earnings_alert_preferences(email)</code>	120s session	Loads prefs via <code>earnings_alert_store</code>
<code>upsert_earnings_alert_preferences(...)</code>	—	Persists prefs; invalidates session cache

7.3 Dedup and fiscal quarter logic

- `fiscal_quarter_ending` stored as `Mon/YYYY` (e.g. `Sep/2025`)
- Quarter derived: Jan–Mar = Q1 ... Oct–Dec = Q4
- Event colors use Coresight palette: Q1 dark blue, Q2 green, Q3 plum, Q4 gold, M&A red (`#D62E2F`)

8. Page load and data fetch

8. Page load and data fetch

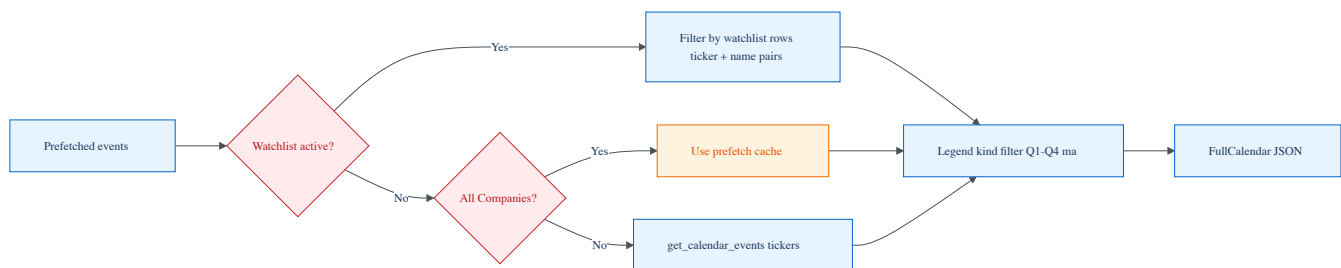


Timing logged as `EC_PAGE_FETCH_*` and `EC_PAGE_RENDER_TOTAL` (see server logs).

9. Filter pipeline

Filters apply **in sequence** (each narrows the in-memory event lists):

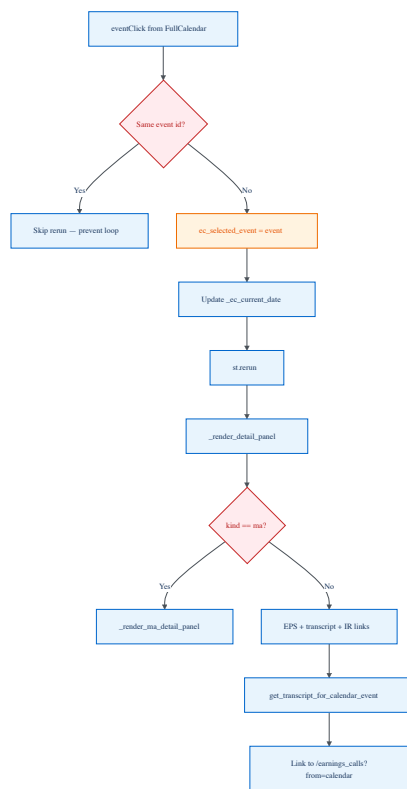
Filters apply in sequence (each narrows the in-memory event lists):



Watchlist disambiguation: For shared tickers (e.g. JD, LULU, TSCO), when both watchlist row and event carry `company_name`, a `(ticker_variant, normalized_name)` match is required.

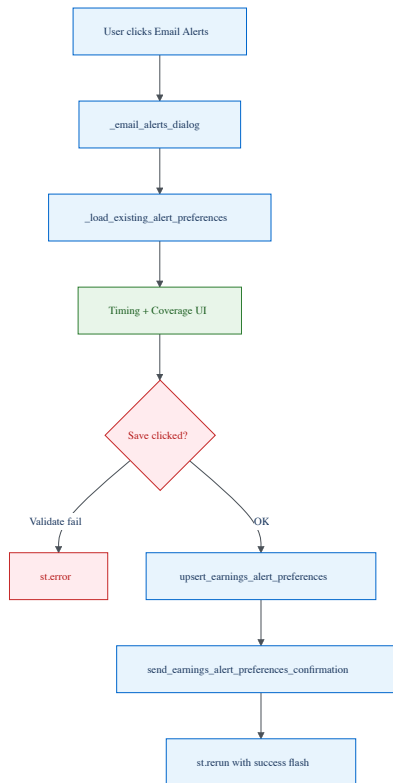
10. Event click and detail flow

10. Event click and detail flow



11. Email alert preferences

11. Email alert preferences



Watchlist encoding: `selection_mode == "watchlist"` stores `tickers_json = [{"watchlist_id": <id>"]`. The background dispatcher (`utils/earnings_alert_dispatcher.py`) resolves tickers at send time.

12. FullCalendar event shape

Earnings events (`_to_fullcalendar`):

```
{
  "id": "12345",
  "title": "Apple Inc Q1",
  "start": "2025-01-30",
  "color": "#005F8F",
  "extendedProps": {
    "kind": "Q1",
    "ticker": "AAPL",
    "fiscal_q": 1,
    "fiscal_year": 2025,
    "eps_actual": 2.18,
    "beat_miss": "beat",
    "ir_website_url": "https://..."
  }
}
```

M&A events use `id` prefix `ma_` and `kind: "ma"`.

13. Error handling

Area	Behavior
Parallel fetch failure	Per-future <code>log_structured_error</code> ; returns empty list / None
No calendar data	<code>st.error("No earnings calendar data found.")</code>
No events after filters	HTML message with hint to re-enable legend chips
Top-level render	<code>st.error("Earnings calendar is temporarily unavailable.")</code>
Alert save failure	<code>st.error(message)</code> from <code>_save_alert_preferences</code>
All exceptions	<code>log_structured_error(..., page="earnings_calendar", ...)</code>

14. Authentication and access control

Check	Status
<code>require_auth()</code>	Cookie hydrate via <code>main.py</code> ; optional page-level enforcement
<code>main.py</code> cookie hydrate	Provides identity when auth active
Email alerts	Requires signed-in user email from <code>auth_data</code> / session
Page Access Control List (ACL)	No dedicated ACL gate — available to all authenticated portal users

15. Performance notes

- **Parallel prefetch** — 4 workers on initial load; subsequent company-specific fetch hits `@st.cache_data` on repository methods
- **Warmup** — `cache_manager.py` pre-warms `get_available_tickers` , `_get_fiscal_year_end_map` , `get_calendar_events` on app boot
- **Calendar remount** — `ec_cal_version` in widget `key` prevents stale `eventClick` state loops
- **Navigation** — `navigation.py` queues page switch if calendar load is mid-run

16. Related files

File	Role
<code>app/data/repository.py</code>	<code>EarningsCalendarRepository</code> — all SQL
<code>app/data/earnings_alert_store.py</code>	Alert preference persistence
<code>app/utils/earnings_alert_dispatcher.py</code>	Scheduled email delivery
<code>app/utils/earnings_alert_email.py</code>	Simple Mail Transfer Protocol (SMTP) confirmation
<code>app/pages/earnings_calls.py</code>	Transcript viewer (deep-link target)
<code>app/components/navigation.py</code>	Nav link with ticker param
<code>app/utils/cache_manager.py</code>	Background warmup

17. Key code references

Function	Lines (approx.)	Purpose
<code>render_page()</code>	1605–2118	Main page orchestration
<code>_to_fullcalendar()</code>	1252–1296	Earnings → FullCalendar JSON
<code>_ma_to_fullcalendar()</code>	1299–1345	M&A → FullCalendar JSON
<code>_render_detail_panel()</code>	1421–1521	Earnings detail card
<code>_render_email_alert_preferences_inner()</code>	880–1206	Alert dialog body
<code>_filter_events_by_watchlist()</code>	1594–1598	Watchlist scope filter